

Rakshak

Multilingual Disaster Alert and SOS Platform

Practical/Viva Explanation Guide

This document explains the full project code, important flows, database design, alert system, SOS system, admin panel, frontend logic, and expected practical questions.

[Node.js](#)[Express.js](#)[MongoDB](#)[Mongoose](#)[EJS](#)[Leaflet Maps](#)[Server Sent Events](#)[Multilingual Alerts](#)

1. Project Overview

Rakshak is a full-stack web application for disaster alert communication and emergency SOS handling. The main problem it solves is that during a disaster, people need fast alerts in a language they understand, clear safety guidance, nearby safe resources, and a simple way to ask for help.

The project has two main user roles:

- **Normal user:** registers, logs in, views local disaster alerts, listens to alerts, checks weather, marks safe/need help, sends SOS, and helps nearby SOS requests.
- **Admin:** manages alerts, resources/safe zones, SOS requests, verification status, and dashboard statistics.

One-line explanation for viva: Rakshak is a Node.js, Express, MongoDB and EJS based disaster management platform that provides multilingual alerts, official feed integration, weather information, SOS creation with GPS, real-time SOS updates, and admin verification.

2. Technology Stack

Part	Technology	Purpose
Backend	Node.js, Express.js	Creates routes, controllers, sessions, APIs, and server-side rendering.
Frontend	EJS, HTML, CSS, JavaScript	Builds dynamic pages and browser interactions.
Database	MongoDB with Mongoose	Stores users, alerts, SOS records, resources, sessions, and check-in status.
Authentication	express-session, connect-mongo, bcryptjs	Secure login sessions and hashed passwords.
Maps	Leaflet and marker clustering	Shows alerts, safe zones, SOS requests, routes, and check-in location.
External APIs	SACHET RSS, Google Translate endpoint, Google TTS endpoint, Open-Meteo, OSRM, Twilio/webhook optional	Official alerts, translations, voice playback, weather, routes, and SMS fallback.

3. Folder Structure

Folder/File	Meaning
<code>server.js</code>	Main entry point. Starts Express, connects database, configures session, static files, routes, and error pages.
<code>config/</code>	Database connection, supported languages, and state coordinates.
<code>models/</code>	Mongoose schemas for MongoDB collections.
<code>controllers/</code>	Business logic for auth, alerts, SOS, resources, and check-in status.
<code>routes/</code>	Express route definitions that connect URLs to controller functions.
<code>middleware/</code>	Authentication and authorization guards.
<code>services/</code>	Reusable helper services for weather, translation, official alerts, SMS, event bus, and admin seeding.
<code>views/</code>	EJS pages rendered on the server.
<code>public/</code>	CSS, browser JavaScript, manifest, and service worker.
<code>scripts/</code> and <code>seed/</code>	Utility scripts for demo data and local development.

4. Application Startup Flow

4.1 server.js

1. Imports Express, path, dotenv, express-session, database connection, and admin seed service.
2. Loads environment variables using `dotenv.config()`.
3. Connects to MongoDB by calling `connectDB()`.
4. After DB connection, calls `ensureDefaultAdmin()` so the app always has an admin account if one does not exist.
5. Configures EJS as the view engine and sets the `views` folder.
6. Enables form parsing with `express.urlencoded()` and JSON parsing with `express.json()`.
7. Serves static files from `public/`.
8. Configures sessions. Session data is stored inside MongoDB using `connect-mongo`, so login remains stable even if the server restarts.
9. Adds `res.locals.currentUser`, making the logged-in user available in all EJS views.
10. Mounts routes: alerts/home, admin, SOS, status, auth, and TTS.
11. Adds a 404 page for unknown routes.
12. Starts the server on `PORT` from environment or default `3000`.

Viva answer: The first file executed is `server.js`. It prepares middleware, session, database, routes and starts listening. It is the central file that connects the whole MVC structure.

5. Database Connection and Configuration

5.1 config/db.js

This file exports an async `connectDB` function. It reads `MONGO_URI` from the environment. If the variable is missing, it uses local MongoDB: `mongodb://127.0.0.1:27017/disaster_alert_mvp`. If connection succeeds, it logs `MongoDB connected`. If it fails, it logs the error and exits the process.

5.2 config/languages.js

This file defines all supported languages and maps each language to an alert message field. Example: Hindi maps to `messageHindi`, Marathi maps to `messageMarathi`. It also stores voice codes and has `detectLanguageFromHeader` to detect preferred language from browser headers.

5.3 config/stateCoordinates.js

This file stores latitude and longitude for supported Indian states. The app uses these coordinates for map centering, weather API calls, and SOS fallback location when exact GPS is not available.

6. Mongoose Models

6.1 User Model

`User.js` stores user account details. Important fields are name, email, phone, password, state, city, language, role, availability, emergency contact name and phone. Password is stored as a bcrypt hash, not plain text. Role can be `user` or `admin`.

6.2 Alert Model

`Alert.js` stores disaster alerts created by admins. It stores title, disaster type, state, city, severity, optional latitude/longitude, multilingual messages, and precautions. Severity is restricted to High, Medium, or Low. Timestamps automatically add `createdAt` and `updatedAt`.

6.3 SOS Model

`SOS.js` stores emergency help requests. It supports registered or anonymous-style user name values, contact number, state, city, distress message, GPS latitude/longitude, location accuracy, source, status, urgency, assigned volunteer, verification status, verified admin, and admin notes.

Important status values are `Pending`, `In Progress`, and `Resolved`. Verification values are `Unverified`, `Verified`, `Needs Review`, and `False Alarm`.

6.4 Resource Model

`Resource.js` stores safe zones such as shelters, hospitals, water, food, and medical support points. It includes location, capacity, contact, address, and active/inactive status.

6.5 SafeCheck Model

`SafeCheck.js` stores a user's response for a specific alert, such as `Safe` or `Need Help`.

6.6 CheckInStatus Model

`CheckInStatus.js` stores the latest global status of each user. It uses `userId` as unique, so one user has one current check-in record. Admin can view safe users and users needing help on dashboard maps.

7. Authentication Flow

7.1 Register

The register page is handled by `authController.register`. It checks password and confirm password, validates minimum password length, checks duplicate email, hashes the password with `bcrypt`, creates the user, stores selected details in `req.session.user`, and redirects based on role.

If the user enters the correct admin code from `ADMIN_SECRET` or default `ADMIN123`, the role becomes admin. Otherwise the role is normal user.

7.2 Login

`authController.login` finds the user by email, compares the entered password with the `bcrypt` hash, stores user info in session, then redirects admin to `/admin` and user to localized `/alerts`.

7.3 Profile

The profile controller lets the user update name, email, phone, state, city, language, and emergency contact. After update, the session is refreshed with latest values.

7.4 Logout

Logout destroys the session and redirects to home.

7.5 Middleware

`requireAuth` protects logged-in pages. It returns JSON for AJAX requests, ends event streams with 401, or redirects normal browser requests to login. `requireAdmin` allows only admin users.

`redirectIfAuthenticated` prevents logged-in users from opening login/register pages again.

8. Alert System Flow

8.1 Admin Creates Alert

1. Admin opens `/admin/alerts/new`.
2. The route calls `alertController.newAlertForm`.
3. Admin fills title, disaster type, state, severity, multilingual messages, location, and precautions.
4. Form submits to `POST /admin/alerts`.
5. `createAlert` parses latitude/longitude and saves the alert in MongoDB.
6. User can now see the alert on `/alerts` if the state matches.

8.2 User Views Alerts

`alertController.userAlertList` is one of the most important functions. It decides selected state, city and preferred language. Then it loads multiple things in parallel: local alerts from MongoDB, official alerts from SACHET, resources, weather, and current check-in status.

For each local alert, it selects the correct language field using `getLocalizedMessage`. It also translates precautions using `translationService.translateText` and creates action guidance using `alertGuidanceService.buildGuidance`.

The controller also chooses a `popupAlert`. It sorts by severity and latest date, so the most important alert can be shown as an instant alert popup.

8.3 Official Alerts

`officialAlertService.fetchOfficialAlerts` calls the SACHET NDMA RSS feed for the selected state. It parses XML using `xml2js`, removes HTML from descriptions, normalizes items, and removes duplicates. If the feed fails, it returns an empty list so the app still works.

8.4 Translation

`translationService.js` uses a Google Translate endpoint. It maps language names to codes such as `hi`, `mr`, `ta`. It uses a memory cache so the same text is not translated repeatedly. If translation fails, it returns the original English text.

8.5 Voice/TTS

`ttsRoutes.js` receives text and language, limits text to 200 characters for safety, calls Google's translate TTS endpoint, and sends back MP3 audio. In the browser, `main.js` splits long alert text into chunks and plays each audio chunk one by one.

9. SOS System Flow

9.1 User Sends SOS

1. User opens alerts page and presses the SOS form button.
2. `public/js/main.js` tries to capture high-accuracy GPS using browser geolocation.
3. If GPS works, hidden fields are filled with latitude, longitude, accuracy, and source `gps`.
4. If GPS fails, user can still submit SOS with state/city fallback.
5. Form submits to `POST /sos`.
6. `sosController.createSOS` saves SOS in MongoDB.
7. It publishes a real-time SOS event using `publishSOSChange`.
8. It calls `sendSOSFallbackSMS` in background if SMS configuration exists.
9. User sees `sos-success.ejs` and can start live location sharing.

9.2 Live Location Update

After SOS success, `public/js/sos-live.js` can repeatedly watch the user's GPS. It sends location updates to `POST /sos/:id/location`. The server checks ownership before updating. It only allows the owner of that SOS to update location. If the request is resolved, update is blocked.

9.3 Real-time SOS Updates

`sosEventBus.js` creates an in-memory EventEmitter. When a SOS is created, accepted, resolved, deleted, or updated, the server publishes an event. `streamSOSUpdates` opens a Server Sent Events stream at `/sos/api/stream`. Admins can see all events; users see only events from their state.

9.4 Nearby SOS Page

`/sos/nearby` shows nearby requests. Browser code in `public/js/sos.js` fetches `/sos/api/requests`, renders SOS cards, displays requests on Leaflet map, allows volunteers to accept/resolve, allows owner to cancel, and can draw route from responder location to victim location using OSRM route API.

9.5 Admin Verification

Admin opens `/admin/sos`. `adminSOSList` loads all SOS records and builds a verification profile. It checks GPS confidence, contact availability, message quality, duplicate recent requests, and responder assignment. This helps admin decide whether SOS is genuine, needs review, or is false alarm.

9.6 SMS Fallback

`smsService.js` collects recipients from `ADMIN_ALERT_PHONE` and user's emergency contact. It supports Twilio or generic webhook provider. If SMS variables are not configured, SOS still saves and real-time web alerts still work.

10. Admin Dashboard

`alertController.adminDashboard` uses `Promise.all` to collect many dashboard values efficiently:

- Total alert count and SOS count.
- Latest alerts and latest SOS requests.
- High, medium, and low severity alert counts.
- Pending and resolved SOS counts.
- Total safe users and users needing help.
- Latest check-ins and check-in map data.

The dashboard helps admin monitor disaster activity, urgent SOS requests, and user safety status.

11. Resource/Safe Zone Management

`resourceController.js` manages safe zones and emergency resources. Admin can add, edit, delete and list resources. Each resource has type, state, city, coordinates, capacity, phone, and active status. On user alert page, active resources of selected state are loaded and shown on the map as safe zones.

Viva answer: Resources represent practical help points like shelters and hospitals. They are stored in MongoDB and shown on the same map as alerts so users can find nearby help.

12. Check-in / Mark Safe System

There are two related concepts:

- `SafeCheck` stores a response for a particular alert.
- `CheckInStatus` stores the latest global safety status of a user.

`statusController.checkIn` receives `safe` or `need_help` from the browser. It saves location if available. It uses `findOneAndUpdate` with `upsert`, so it updates existing status or creates a new one.

Admin dashboard uses these records to show how many users are safe and how many need help.

13. Frontend JavaScript Explanation

13.1 public/js/main.js

This is the main browser script. It handles dark mode, accessibility mode, toast messages, voice playback, maps, AJAX filtering, WhatsApp sharing, copy-to-clipboard, GPS capture, SOS submission, check-in buttons, and service worker registration.

Important functions

- `getFreshGpsLocation` : watches GPS and returns best location within timeout.
- `captureCurrentSosLocation` : fills SOS hidden fields with fresh GPS.
- `setupAlertMap` : creates Leaflet map for alerts and safe zones.
- `speakText` : plays translated alert audio through `/tts` .
- `handleFilterSubmit` : updates alert page with AJAX when filter changes.
- `setStatusCheckIn` : sends safe/need-help check-in to server.

13.2 public/js/sos.js

This script powers the nearby SOS page. It loads SOS requests, renders cards, updates the map, calculates distances, shows browser notifications, connects to real-time SSE stream, falls back to polling every 30 seconds, caches requests for offline-like viewing, and draws route lines.

13.3 public/js/sos-live.js

This script runs after SOS is sent. It lets the victim start/stop live GPS sharing. It avoids sending too many updates by checking time and movement distance, then calls `/sos/:id/location` .

13.4 public/sw.js and manifest.json

`manifest.json` gives app name, theme color, icons, and standalone display behavior. `sw.js` caches important assets and handles notification clicks by opening `/sos/nearby` .

14. Views / EJS Pages

View	Purpose
<code>views/home.ejs</code>	Landing/home page with platform introduction and navigation.
<code>views/auth/register.ejs</code>	User/admin registration form.

<code>views/auth/login.ejs</code>	Login form.
<code>views/auth/profile.ejs</code>	Profile update form.
<code>views/alerts/index.ejs</code>	Main user dashboard for alerts, weather, map, resources, check-in, and SOS form.
<code>views/alerts/show.ejs</code>	Detailed alert page with localized message and guidance.
<code>views/admin/dashboard.ejs</code>	Admin dashboard analytics and check-in map.
<code>views/admin/alerts.ejs</code>	Admin alert management list.
<code>views/admin/new-alert.ejs</code>	Create/edit alert form.
<code>views/admin/sos.ejs</code>	Admin SOS monitoring and verification page.
<code>views/admin/resources.ejs</code>	Safe zone/resource list.
<code>views/admin/new-resource.ejs</code>	Create/edit resource form.
<code>views/sos/nearby.ejs</code>	Nearby SOS volunteer/help page.
<code>views/sos-success.ejs</code>	Confirmation page after SOS submission.
<code>views/preparedness.ejs</code>	Emergency preparedness guide.
<code>views/partials/header.ejs</code> , <code>footer.ejs</code>	Reusable layout parts.
<code>views/404.ejs</code> , <code>500.ejs</code>	Error pages.

15. Routes Summary

Route	Controller/Action	Use
GET /	homePage	Home page.
GET /alerts	userAlertList	User alert dashboard.
GET /alerts/:id	alertDetails	Alert detail page.
POST /alerts/:id/safe	markAsSafe	Mark safe for alert.
GET/POST /auth/register	showRegister/register	Registration.
GET/POST /auth/login	showLogin/login	Login.
GET/POST /auth/profile	showProfile/updateProfile	Profile.
POST /auth/logout	logout	Logout.
POST /sos	createSOS	Create SOS.
GET /sos/nearby	nearbySOS	Nearby SOS page.
GET /sos/api/requests	getNearbyRequests	Nearby SOS JSONAPI.
GET /sos/api/stream	streamSOSUpdates	Server Sent Events live updates.
POST /sos/:id/accept	acceptRequest	Volunteer accepts SOS.
POST /sos/:id/resolve	resolveRequest	Resolve SOS.
POST /sos/:id/cancel	cancelRequest	Owner cancels SOS.
POST /sos/:id/location	updateSOSLocation	Victim updates live GPS.
POST /status/checkin	checkIn	Save safe/need-help status.
GET /tts	ttsRoutes	Return MP3 voice for alert text.
/admin/*	Admin controllers	Dashboard, alerts, resources, SOS verification.

16. How to Explain Major Features in Practical

16.1 How SOS Works

When user clicks SOS, browser JavaScript requests GPS. If allowed, it stores latitude, longitude, accuracy, and source in hidden form fields. The Express route `POST /sos` calls `createSOS`. The controller validates coordinates, creates an SOS document in MongoDB, publishes a real-time event to connected clients, tries SMS fallback if configured, and renders success page. Admin and nearby users receive updates through SSE and API refresh.

16.2 How Alert Is Coming

There are two sources of alerts. First, admin-created alerts are stored in MongoDB and fetched by state. Second, official alerts are fetched from SACHET NDMA RSS feeds. The alert controller combines both sources, translates content if needed, adds weather and guidance, then renders `alerts/index.ejs`.

16.3 How Multilingual Support Works

The alert model stores separate fields for different languages. The selected language comes from query parameter, session, user profile, or browser header. The controller chooses the matching message field. For dynamic content like official alerts and guidance, the translation service calls Google Translate and caches the result.

16.4 How Real-time Updates Work

The server uses Node's EventEmitter in `sosEventBus.js`. When SOS changes, it emits an event. The SSE route keeps an HTTP connection open and writes events to the browser. The browser listens using `EventSource`. If SSE has a problem, the frontend still polls every 30 seconds.

16.5 How Admin Detects False SOS

The system does not automatically delete SOS. It gives admin verification signals: GPS confidence, contact availability, generic message, duplicate nearby/recent requests, and responder assignment. Admin can mark as Verified, Needs Review, or False Alarm. False alarm can then be resolved or deleted.

17. Expected Viva Questions and Answers

Basic Questions

1. **What is your project?**

Rakshak is a multilingual disaster alert and SOS platform for users and admins. It shows local alerts, official alerts, weather, safe zones, and lets users send SOS with GPS.

2. **Which architecture did you use?**

I used MVC style: models for database schemas, controllers for logic, routes for URLs, views for EJS pages, services for reusable external/API logic.

3. **Why MongoDB?**

MongoDB is flexible for storing user profiles, alerts, SOS requests, and dynamic fields like multilingual messages and location data. Mongoose provides schema validation.

4. **How do you secure passwords?**

I use bcryptjs. During registration password is hashed with salt rounds, and during login bcrypt compares entered password with stored hash.

5. **How are sessions managed?**

I use express-session and connect-mongo. Session ID is stored in browser cookie and session data is stored in MongoDB.

SOS Questions

1. **How does SOS capture location?**

The frontend uses browser geolocation with high accuracy. Coordinates and accuracy are submitted to backend hidden fields. If GPS fails, SOS still saves with state/city.

2. **What happens after SOS is saved?**

It is stored in MongoDB, an SSE event is published, optional SMS fallback is triggered, and the user gets success page.

3. **Can anyone update SOS location?**

No. The backend checks ownership using session user ID. Only the owner can update live location.

4. **How does volunteer help work?**

Nearby SOS page fetches requests from same state. A user cannot accept their own SOS. A volunteer can accept pending requests and mark them resolved.

5. **What if GPS is unavailable?**

The system still sends SOS using selected state/city and map fallback coordinates. It clearly marks fallback as approximate.

Alert Questions

1. How are alerts filtered?

User selected state is used to query MongoDB alerts and official RSS feed. City filtering is prepared but currently not enforced.

2. How do you choose alert language?

Priority is query parameter, logged-in user's language, session language, browser Accept-Language header, then English.

3. What is popup alert?

The controller sorts alerts by severity and latest date. The highest severity/latest alert becomes popup alert.

4. Where do official alerts come from?

From SACHET NDMA RSS feed. The app fetches XML, parses with xml2js, cleans HTML, and shows official feed items.

5. How voice alert works?

Frontend sends text and language to `/tts`. Backend calls Google TTS endpoint and returns MP3 audio.

Admin Questions

1. How is admin protected?

All admin routes use `requireAdmin`. If user is not logged in, redirect to login. If role is not admin, redirect to alerts.

2. How is default admin created?

After DB connection, `ensureDefaultAdmin` checks if admin email exists. If not, it creates one with hashed password.

3. What does admin dashboard show?

Total alerts, SOS counts, severity counts, pending/resolved SOS, latest records, safe users, need-help users, and map data.

4. How are false SOS requests handled?

Admin can mark verification status as False Alarm. Delete is allowed when it is false alarm or admin confirms false alarm.

18. File-by-File Quick Explanation

File	Explain Like This
<code>package.json</code>	Lists project name, scripts, dependencies and dev dependency nodemon.
<code>server.js</code>	Main Express server setup, middleware, sessions, routes, 404, port listen.

<code>config/db.js</code>	MongoDB connection helper.
<code>config/languages.js</code>	Supported languages, message fields, browser language detection.
<code>config/stateCoordinates.js</code>	State center coordinates for maps/weather/fallback.
<code>models/User.js</code>	User schema with account, role, language, location and emergency contact.
<code>models/Alert.js</code>	Alert schema with severity, messages in many languages, location and precautions.
<code>models/SOS.js</code>	SOS schema with victim, location, status, volunteer, verification and admin note.
<code>models/Resource.js</code>	Safe zone/resource schema.
<code>models/SafeCheck.js</code>	User's safe/need-help response for a particular alert.
<code>models/CheckInStatus.js</code>	Latest safety status of each user.
<code>controllers/authController.js</code>	Register, login, logout and profile update logic.
<code>controllers/alertController.js</code>	Home, admin dashboard, alert CRUD, user alert list, alert details, safe marking.
<code>controllers/sosController.js</code>	Create SOS, update location, admin SOS list, verification, nearby API, SSE, accept/resolve/cancel.
<code>controllers/resourceController.js</code>	Admin CRUD for resources/safe zones.
<code>controllers/statusController.js</code>	Check-in API for safe/need-help status.
<code>middleware/authMiddleware.js</code>	Route guards for login and admin access.
<code>routes/*.js</code>	URL definitions that call controller methods.
<code>services/translationService.js</code>	Translation with cache and official alert translation.
<code>services/officialAlertService.js</code>	SACHET RSS fetch, XML parse, clean, normalize and deduplicate.
<code>services/weatherService.js</code>	Open-Meteo weather fetch, caching and fallback.
<code>services/alertGuidanceService.js</code>	Prepared action guidance based on disaster type and severity.
<code>services/sosEventBus.js</code>	Real-time SOS event emitter.
<code>services/smsService.js</code>	Optional SMS sending through Twilio or webhook.
<code>services/adminSeedService.js</code>	Creates default admin if missing.

<code>public/js/main.js</code>	Main browser interactions: maps, TTS, SOS GPS, check-in, filters, sharing, service worker.
<code>public/js/sos.js</code>	Nearby SOS live page, map, route, notifications, SSE and polling.
<code>public/js/sos-live.js</code>	Live victim location sharing after SOS.
<code>public/css/style.css</code>	Complete styling for layouts, cards, maps, forms, dashboard and responsive design.
<code>public/sw.js</code>	Service worker caching and notification click handling.
<code>public/manifest.json</code>	PWA metadata.
<code>seed/seed.js</code> , <code>scripts/seed*.js</code>	Demo data helpers.

19. Practical Demo Script

1. Start MongoDB and run `npm run dev`.
2. Open `http://localhost:3000`.
3. Register a user with state and preferred language.
4. Log in and show the alerts page.
5. Change language and show translated alert/guidance.
6. Click listen to demonstrate text-to-speech.
7. Show alert map with alert markers and safe zones.
8. Use check-in buttons to mark Safe or Need Help.
9. Send SOS. Explain GPS permission and fallback behavior.
10. Open admin login and show dashboard/SOS page.
11. Show SOS verification, status update and map location.
12. Open nearby SOS page and show accept/resolve flow.

Important practical note: Exact GPS works best on HTTPS or localhost. On deployed sites, use the HTTPS URL. If permission is blocked, browser settings must allow location.

20. Strengths, Limitations, and Future Scope

Strengths

- Full-stack implementation with real database.
- Role-based admin and user flows.
- Multilingual disaster alerts.
- Official alert feed integration.
- SOS with GPS and fallback.
- Real-time updates using SSE.
- Safe zone maps and weather information.

Limitations

- SMS works only when Twilio or webhook environment variables are configured.
- Translation/TTS depends on external Google endpoints.
- SSE event bus is in-memory, so for multiple server instances a shared pub/sub system would be better.

- City filtering is present in structure but not fully enforced in alert/resource queries.

Future Scope

- Add push notifications for alerts and SOS at larger scale.
- Add verified government API integration beyond RSS.
- Add admin analytics charts and export reports.
- Add volunteer role separately from user/admin.
- Add offline-first alert storage for disaster network failures.
- Add multi-server real-time updates using Redis Pub/Sub.

21. Final Short Explanation to Memorize

My project Rakshak is a multilingual disaster alert and SOS platform. It is built using Node.js, Express.js, MongoDB, Mongoose, EJS, JavaScript, and Leaflet maps. Users can register, choose language and location, view local alerts, official SACHET alerts, weather, safety guidance, and safe zones. They can listen to alerts using text-to-speech, mark themselves safe or needing help, and send SOS with GPS location. Admin can manage alerts, resources, and SOS requests, verify false alarms, and monitor dashboard statistics. The project uses sessions for authentication, bcrypt for password security, MongoDB for data storage, external APIs for translation, weather, official alerts and routing, and Server Sent Events for real-time SOS updates.